

CONTAINER ORCHESTRATION ASSIGNMENT

StreamingApp on Kubernetes

Package, deploy, expose, and scale a 5-service streaming platform with Kubernetes, Helm, and Ingress.



Kubernetes



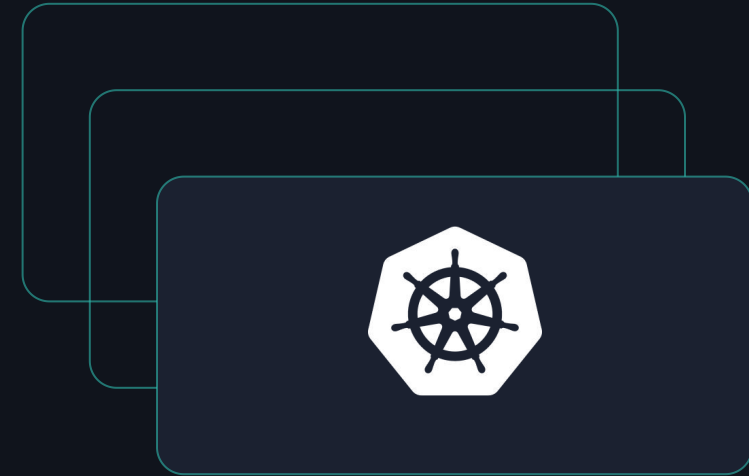
Helm



Ingress



Docker



Learning Objectives

By the end of this assignment, you will be able to:



Containerize a multi-service app

Build and version Docker images for 4 Node.js microservices and a React/Nginx frontend.



Model the system in Kubernetes

Translate docker-compose services into Deployments, Services, ConfigMaps, and Secrets.



Package it as a Helm chart

Templatize manifests with values.yaml so the whole stack installs with one command.



Expose it with Ingress

Route path- and host-based traffic to five backends behind a single entry point.



Scale and update safely

Configure replicas, resource limits, and rolling updates without downtime.



Verify cluster health

Use liveness/readiness probes and smoke tests to confirm the platform actually works.

Architecture Recap

Five services and a shared database — this is what you're about to orchestrate.



authService

Registration, login, JWT issuance

:3001



streamingService

Video catalogue, S3 playback, public API

:3002



adminService

Asset management & signed uploads

:3003



chatService

WebSocket + REST live chat

:3004



frontend

React SPA, served via Nginx

:3000 → 80



MongoDB

Shared instance (:27017) backing every service's data models.

In this assignment: a StatefulSet + PersistentVolumeClaim, or a managed DocumentDB / Atlas endpoint.

Prerequisites & Tooling

Set these up before Task 1 — every later step assumes they work.



Docker Desktop / Engine

To build & tag each service image locally.



Docker Hub account

Public repo to push images (versioned 1.0.x).



A Kubernetes cluster

Minikube/Kind locally, or an EKS/AKS/GKE cluster.



kubectl CLI

Configured against your cluster context.



Helm 3

To template and install the chart you'll author.



Ingress controller

e.g. ingress-nginx, enabled on your cluster.

TASK 1

Containerize Every Service

- 1 Each of the 5 services already ships a Dockerfile — reuse them; do not rewrite the app.
- 2 Build one image per service: authService, streamingService, adminService, chatService, frontend.
- 3 Frontend uses a multi-stage build (Node → Nginx) — keep the final stage slim.
- 4 Tag every image with your Docker Hub username and version, e.g. <user>/streaming-auth:1.0.0.
- 5 Push all 5 images to Docker Hub before writing any Kubernetes manifest.

BUILD & PUSH

```
docker build -t $USER/streaming-auth:1.0.0 \
  backend/authService
docker build -t $USER/streaming-stream:1.0.0 \
  -f backend/streamingService/Dockerfile backend
docker build -t $USER/streaming-admin:1.0.0 \
  -f backend/adminService/Dockerfile backend
docker build -t $USER/streaming-chat:1.0.0 \
  -f backend/chatService/Dockerfile backend
docker build -t $USER/streaming-frontend:1.0.0 \
  frontend

docker push $USER/streaming-<service>:1.0.0
```

TASK 2

Write the Kubernetes Manifests

One Deployment + Service pair per component, plus shared config objects.



Deployments

5x — one per service, referencing your pushed image & tag.



ClusterIP Services

Stable internal DNS names for pod-to-pod traffic.



ConfigMap

Non-secret env vars: PORT, CLIENT_URLS, AWS_REGION, MONGO_URI host.



Secret

JWT_SECRET, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY.



StatefulSet + PVC

MongoDB with persistent storage, exposed as mongo.<ns>.svc.



Probes

readinessProbe / livenessProbe on each Deployment's HTTP port.

TASK 3

Package It as a Helm Chart

Turn Task 2's manifests into templates so image tags, replica counts, and env values are all configurable from one file.

CHART LAYOUT

```
streamingapp/  
├─ Chart.yaml  
├─ values.yaml  
└─ templates/  
    ├─ auth-deployment.yaml  
    ├─ auth-service.yaml  
    ├─ streaming-deployment.yaml  
    ├─ streaming-service.yaml  
    ├─ admin-deployment.yaml  
    ├─ chat-deployment.yaml  
    ├─ frontend-deployment.yaml  
    ├─ mongo-statefulset.yaml  
    ├─ configmap.yaml  
    ├─ secret.yaml  
    └─ ingress.yaml
```

values.yaml (excerpt)

```
services:  
  auth:  
    image: <user>/streaming-auth  
    tag: "1.0.0"  
    replicas: 2  
    port: 3001  
  frontend:  
    image: <user>/streaming-frontend  
    tag: "1.0.0"  
    replicas: 2  
  
mongo:  
  storageSize: 5Gi  
  
ingress:  
  host: streamingapp.local
```

TASK 4

Expose Traffic with Ingress

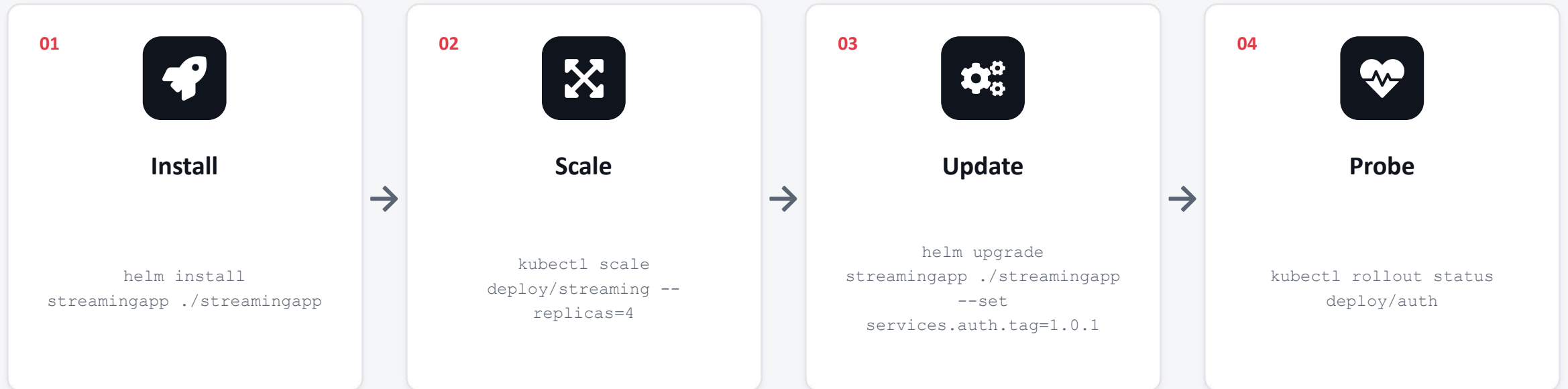
One external host, five internal destinations — route by path.

Ingress Path	Backend Service	Port	Handles
/	frontend-svc	80	React SPA
/api/auth	auth-svc	3001	Login, register, JWT
/api/streaming	streaming-svc	3002	Catalogue & playback
/api/admin	admin-svc	3003	Uploads & curation
/api/chat + ws	chat-svc	3004	Socket.IO chat

Enable TLS via a cert-manager-issued Secret referenced in the Ingress spec, or keep HTTP-only for local clusters.

TASK 5

Deploy, Scale & Update



Zero-downtime requirement

Set `maxUnavailable: 0` and `maxSurge: 1` on the RollingUpdate strategy for every Deployment, and wait for `kubectl rollout status` to succeed before demoing.

TASK 6

Verify & Smoke Test

A cluster with all pods Running isn't proof the platform works — confirm end to end.



All pods Running & Ready

kubectl get pods shows every Deployment at desired replica count.



Register & log in

Create an account through the Ingress host and receive a valid JWT.



Upload via admin

Push a small video + thumbnail through the admin dashboard.



Playback from browse

Confirm the uploaded video streams from the catalogue page.



Live chat broadcast

Messages sent in one browser tab appear in a second tab.



Survives a pod restart

Delete a pod and confirm the Deployment self-heals with no user impact.

Deliverables & Submission



Public GitHub repo containing your Helm chart (Chart.yaml, values.yaml, templates/).



Docker Hub links for all 5 pushed images, tagged 1.0.x.



A README with exact install steps: helm install ... and how to reach the app via Ingress.



Screenshot or short clip of kubectl get pods,svc,ingress -A showing everything Running.



Screenshot of the running app: successful login, one upload, and chat working across two tabs.



A one-paragraph note on what you'd change for a production cluster (namespaces, TLS, HPA).

Evaluation Rubric

Criterion	What we're checking	Weight
Containerization	All 5 images build clean, tagged & pushed to Docker Hub	20%
K8s Manifests	Deployments, Services, ConfigMap/Secret correctly wired	20%
Helm Chart	Fully templated — no hardcoded values in templates/	20%
Ingress Routing	All 5 paths resolve correctly through one host	15%
Scaling & Updates	Replica scaling + rolling update demonstrated live	15%
Verification	Smoke tests pass; pod self-heal demonstrated	10%

Rubric weights are indicative — your instructor may adjust per batch.